Transfer Learning

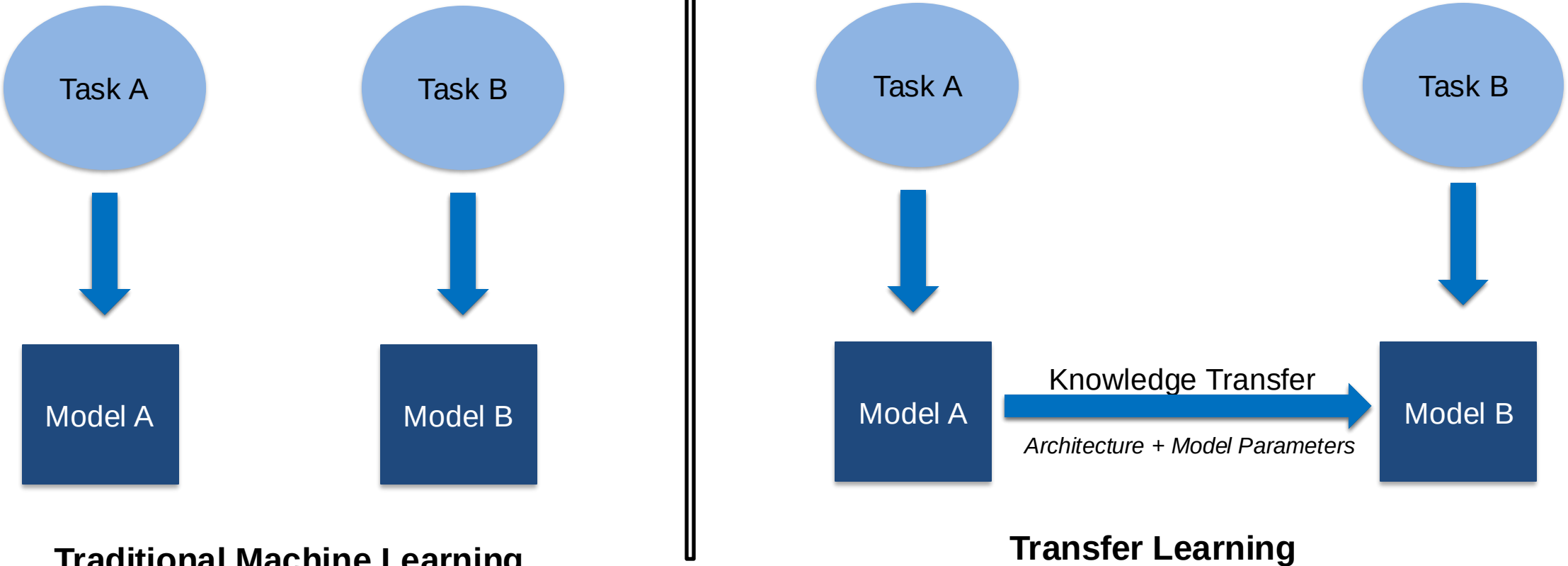
Seshadri Mazumder

Improving a model

Methods to improve a model	What does it do?
More data	Model gets more chance to learn pattern between samples.
Better data	Not all data samples are created equally. Removing bad samples or adding more better samples can improve the model's performance
Augmentation	Increasing diversity of training dataset without collecting more data: performing different transformation operation.
Using Transfer Learning	Take an existing model's pre-learned patterns from one problem and use it for different one by just a little bit of tweaking

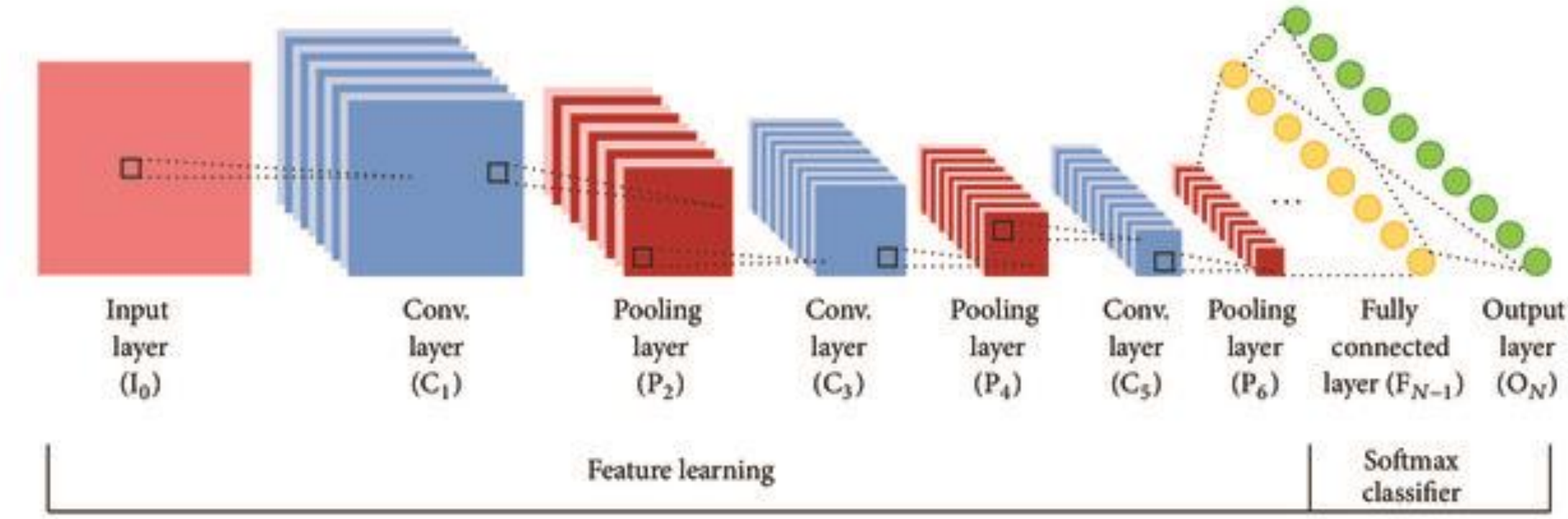
Transferring the knowledge of one model to perform a new task.

How it works


Task A
Task B
Model A
Model B
Traditional Machine Learning
Task A
Task B
Model A
Knowledge Transfer
Architecture + Model Parameters
Model B
Transfer Learning

A CNN typically consists of a:

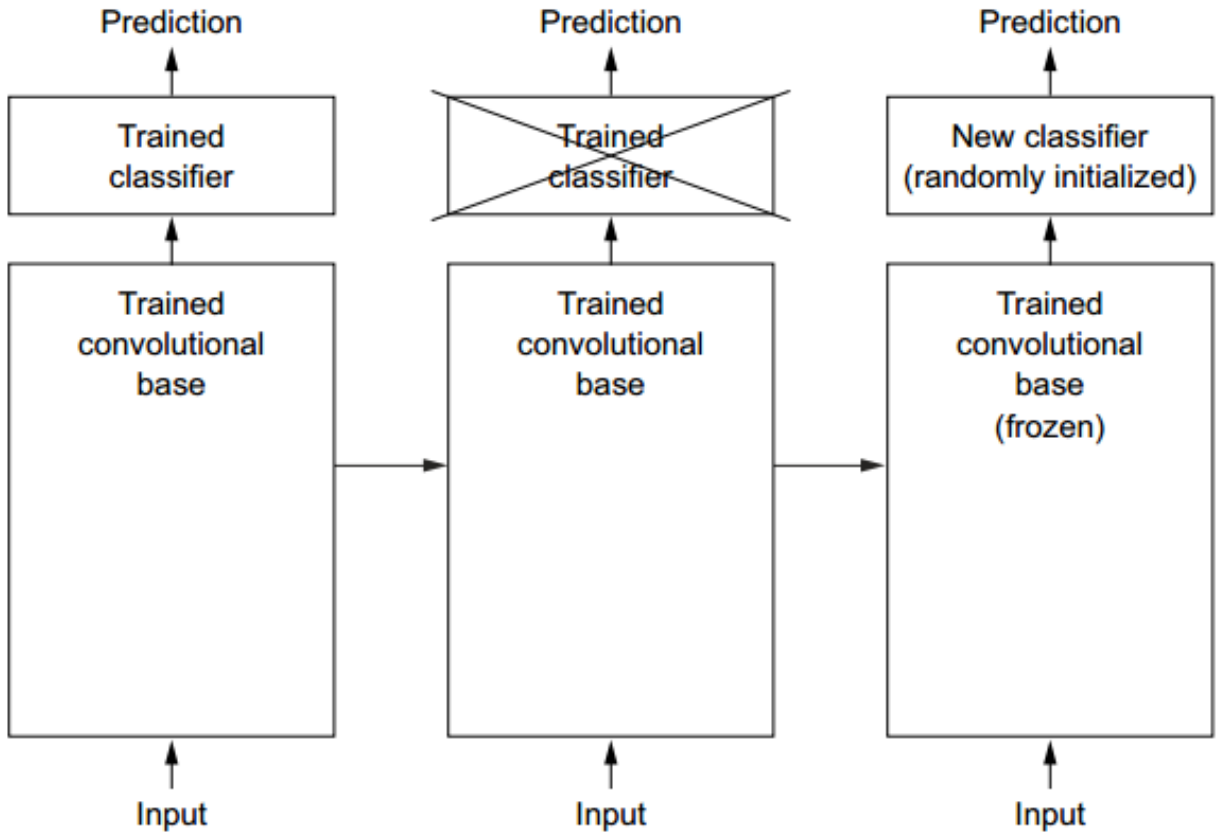
- Convolutional base
- Densely connected classifier



Feature extraction

A CNN typically consists of a:

* Convolutional base
* Densely connected classifier



Key Idea -

* Features are learned by the convolutional base. So reuse it.
* Train a new classifier for your problem

Why To Use

- Unavailability of data
- Long training time to train deep learning models
- More Computational resources

A common and highly effective approach to deep learning on small image datasets is to use a pre-trained model.

If the original dataset is large enough and general enough, the spatial hierarchy of features learned by the pre-trained model can effectively act as a generic model of the visual world, and hence, its features can prove useful for many different computer vision problems even though these new problems may involve completely different classes than those of the original task.

Step involved while using a pre-trained model:

- ❖ Feature extraction
- ❖ Fine-tuning

Transfer Learning Process

1. Start with pre-trained network
2. Partition network into:
    1. Featurizers : identify which layers to keep
    2. Classifiers : identify which layers to replace
3. Re-train classifier layers with new data
4. Unfreeze weights of the later layers and fine-tune the network

Which layers to re-train?

- Depends on the domain
- Start by re-training the last layers (last full-connected and last convolutional)
    - work backwards if performance is not satisfactory

Ways to Fine tune the model

Dataset Size	Dataset Similarity	Recommendation
Large	Very different	Train the model from scratch
Large	Similar	Retain the architecture and initial weights. Customize only last layer for number of classes
Small	Very different	Customize earlier layers and train the classifier
Small	Similar	Don't fine-tune (overfitting)

Pre-trained Models

Different Pre-trained Models

Few …

Network	Year	Parameters	Top-5 accuracy
AlexNet	2012	62M	84.70%
VGGNet	2014	138M	92.30%
Inception	2014	6.4M	93.30%
ResNet-152	2015	60.3M	95.51%

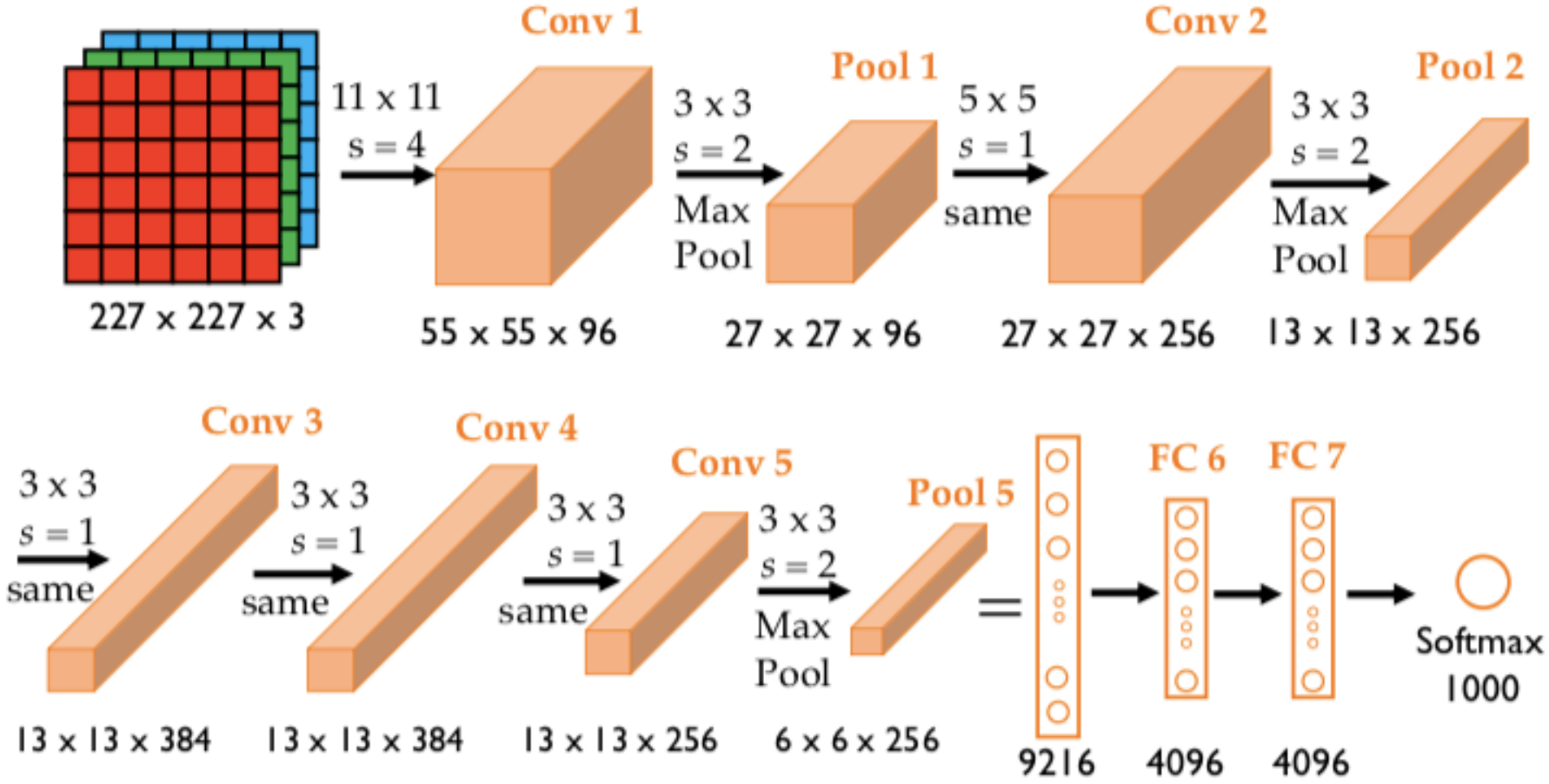
Link for Pre-trained models

→ https://pytorch.org/vision/stable/models.html

→ https://huggingface.co/models

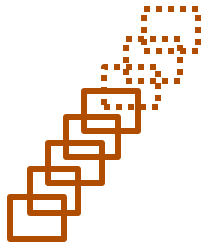
→ https://paperswithcode.com/sota

AlexNet (2012)

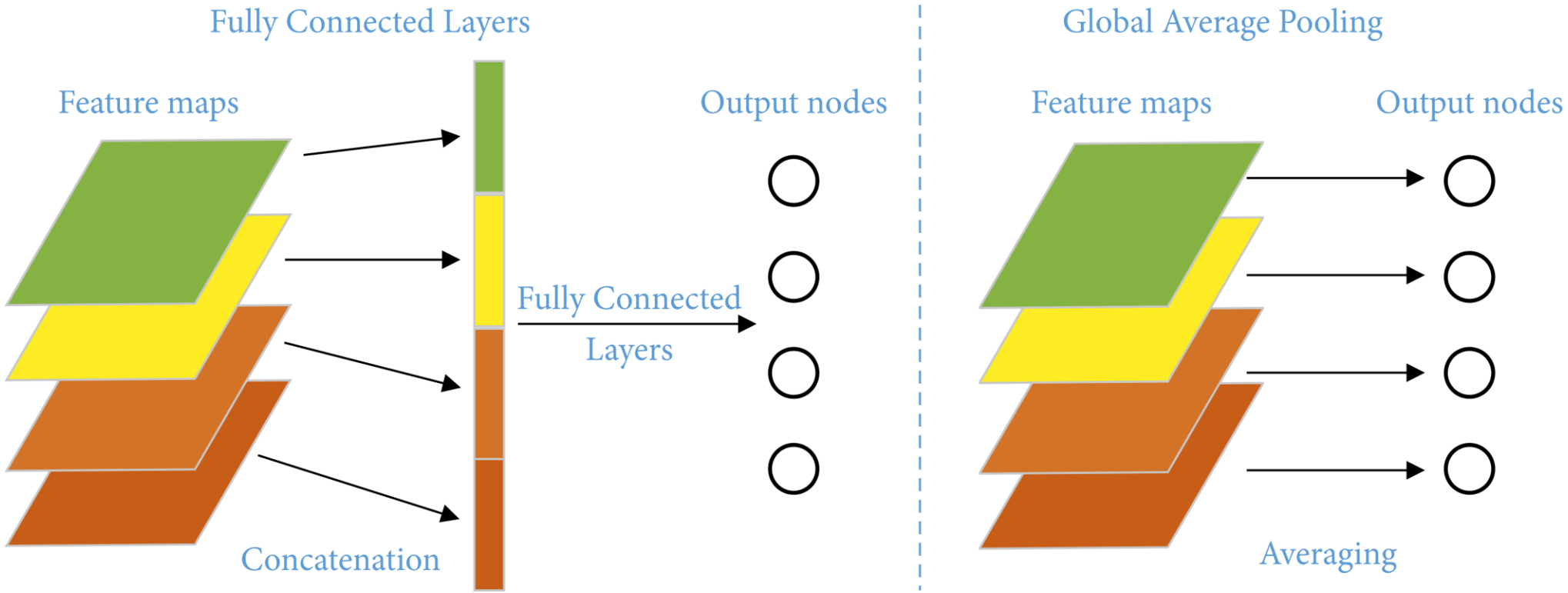


$$Output\ shape\ after\ Conv = \frac{n+2p-f}{s} + 1$$

$$Output\ shape\ after\ Pool = \frac{n-f}{s} + 1$$



Concept of Global Average Pooling at last Convolution Layer



Demo Experiment

Classifying Cats and Dogs using AlexNet and VGG-16 models

- Note: When using a pre-trained model, the data that you pass through it should be transformed in the same way that the data the model was trained on.

Updating Classifier Layer

https://pytorch.org/docs/stable/generated/torch.nn.Sequential.html

```
model.classifier =  torch.nn.Sequential (
                                        )

...

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

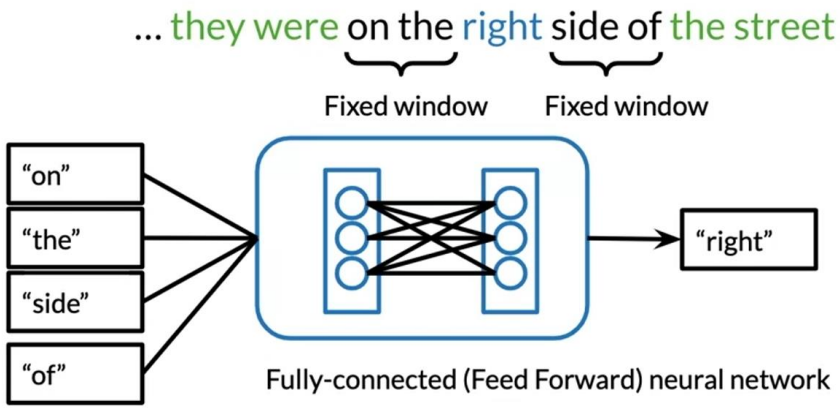
Transfer Learning Concept & Options in NLP

Transfer Learning

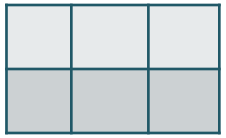
- Feature based
- Fine Tunning

Pre-Train

Continuous Bag of Words:



Or, Skip- gram



Features : Word Embeddings

Input features → Model → Prediction

Train a new model

Transfer Learning Concept & Options in NLP

Language modeling

It is a fundamental technique in NLP that involves predicting the probability of a sequence of words. Essentially, a language model assigns a probability to a sequence of words in a way that reflects how likely the sequence is to occur in a given language. This is crucial for various NLP tasks such as text Classification, text generation, machine translation, and more. Following are the three techniques for language modelling:

1. **Masked Language Modeling (MLM)**: This is used in models like **BERT**, where certain words in a sentence are "masked" or hidden, and the model is trained to predict those missing words based on the surrounding context. This falls under **language modeling** as it involves understanding the context to predict masked words.

2. **Next Sentence Prediction (NSP)**: This is also used in **BERT**, involves predicting whether one sentence logically follows another. While it's related to understanding context in language, it's more of a **sentence-level task** rather than strictly language modeling in the traditional sense. However, it's still under the broad umbrella of pretraining tasks for language understanding.

3. **Next Word Prediction**: This is a classic **language modeling** task where the model predicts the next word in a sequence, often used in models like **GPT**.

These techniques help in building robust language models that can understand and generate human-like text.

Transfer Learning Concept & Options in NLP

BERT Pre-Training

Pre-training datasets

- BookCorpus (800 million words)
- Wikipedia (2500 million words)

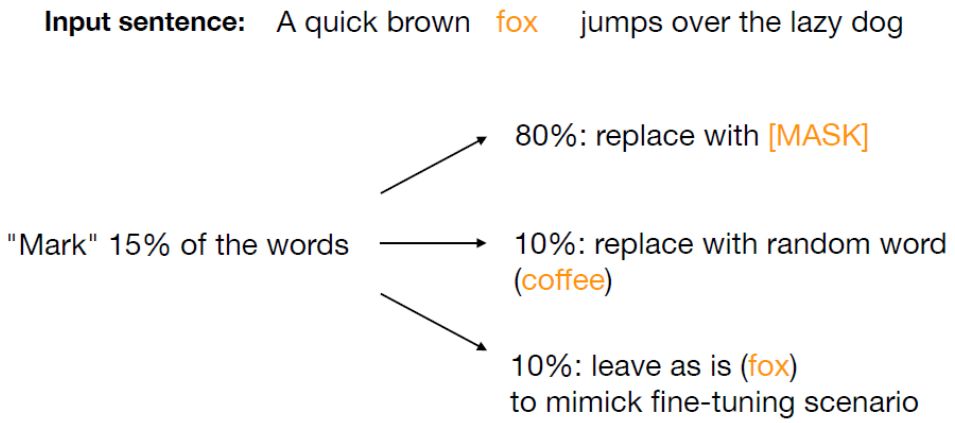
Pre-training tasks

- Masked language model
- Next sentence prediction

Training Samples for MLM

Input Sentence (with `[MASK]`)	Masked Token(s) (Ground Truth)
The cat sat on the **[MASK]**.	mat
She went to the **[MASK]** to buy groceries.	store
He plays the guitar **[MASK]** every evening.	almost
The weather is **[MASK]** today.	sunny
They are having a great time at the **[MASK]**.	beach

Masked language model



Next Sentence Prediction :

Balanced binary classification task (50% IsNext, 50% NotNext)

Input = [CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK] milk [SEP]

Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP] penguin [MASK] are flight ##less birds [SEP]

Label = NotNext

Transfer Learning Concept & Options in NLP

GPT Pre-Training

Pre-training datasets

- WebText Dataset (8 million documents - WebText dataset was created by scraping content from the internet words , excluded Wikipedia: 40 GB) [GPT2-Released on Feb,2019]

Pre-training tasks

- Causal Language Modelling : Next word prediction

Sample dataset curation

Input Sequence	Target Next Word
A	quick
A quick	brown
A quick brown	fox
A quick brown fox	jumps
A quick brown fox jumps	over
A quick brown fox jumps over	the
A quick brown fox jumps over the	lazy
A quick brown fox jumps over the lazy	dog

Input Sequence	Target Next Word
The cat sat	on
The cat sat on	the
The cat sat on the	mat
She went to	the
She went to the	store

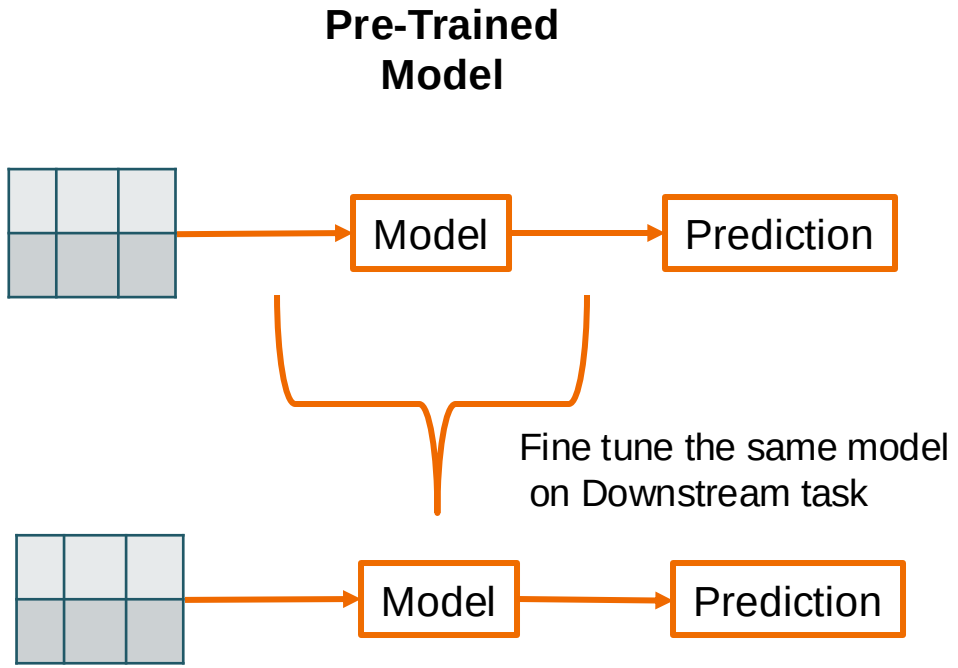
Transfer Learning Concept & Options in NLP

Transfer Learning

- Feature based
- Fine Tunning

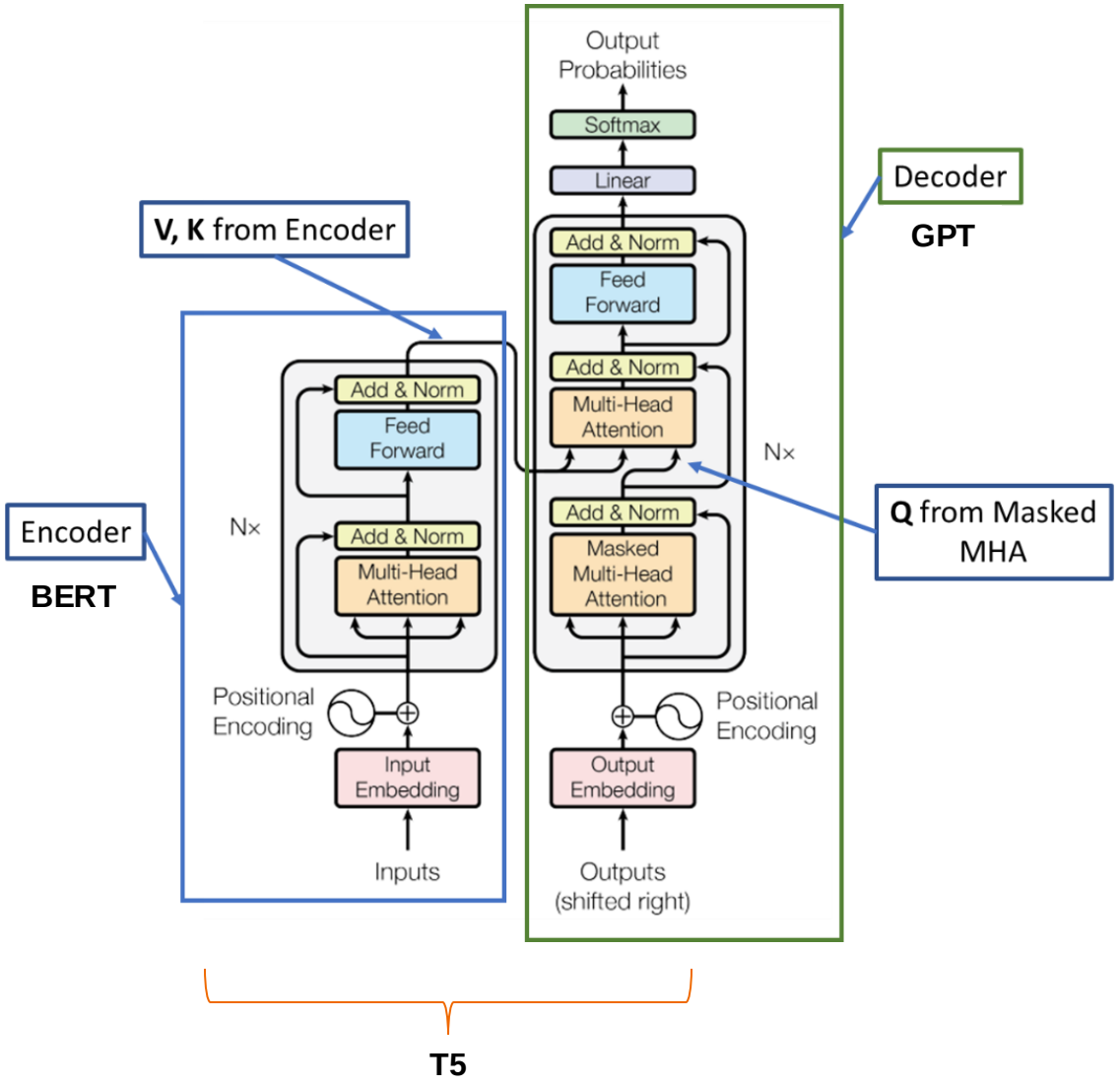
Pre-training task

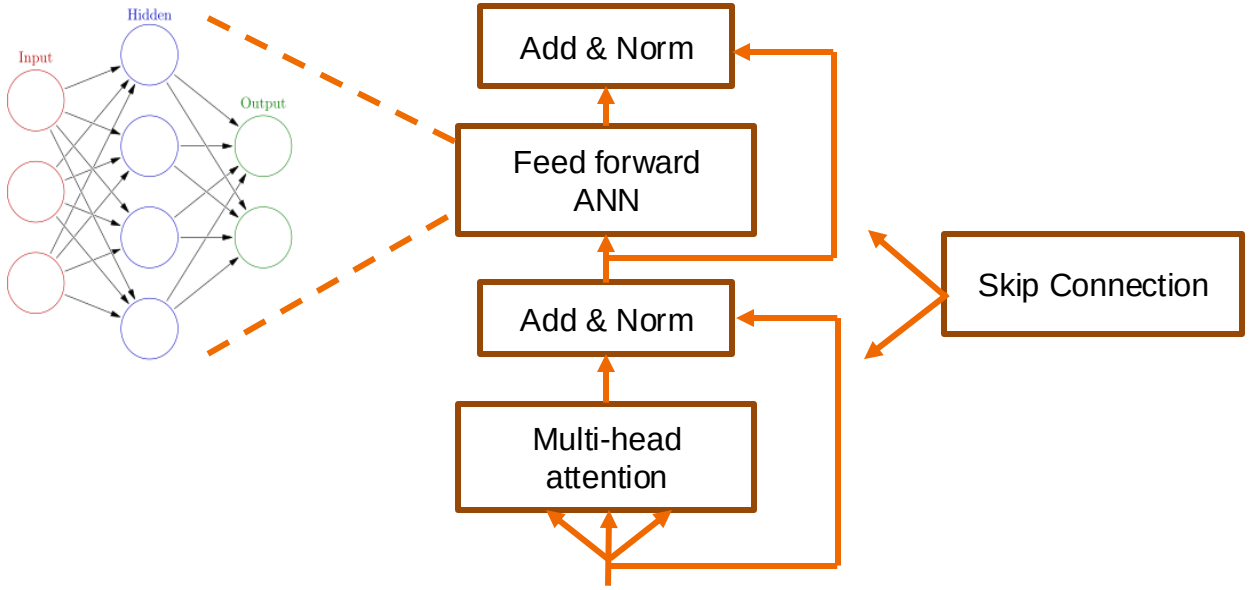
- Language Modeling
- Masked Language Modeling
- Next Sentence Prediction
- Next Word Prediction

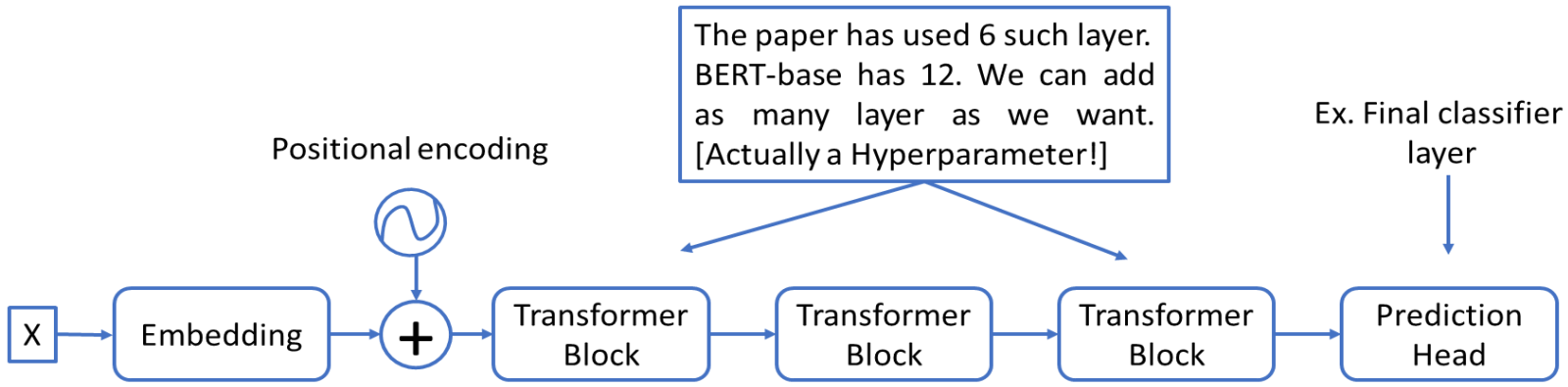


Downstream NLP Tasks :Text Classification, Named Entity Recognition (NER), Question Answering, Text Summarization, Text Generation, Machine Translation, Text-to-Code (Code Generation), Grammar and Spell Correction, ……..……..……..……..

Quick Overview: Transformer Architecture : BERT, GPT, T5







[Link]

Thanks

Questions

Augmentation

The small dataset can cause a high variance estimation of model performance.

To avoid this problem altogether by artificially (and cleverly) producing new data from the already available data.
We perform ****data augmentation****.

Data augmentation takes the approach of generating more training data from existing training samples by augmenting the samples via a number of random transformations that yield a believable-looking image. Common transformations include:

* Flipping the image
* Rotating the image
* Zooming in/out of the image

See some sample images below after augmentation:

Measure Acc@5

Let's say you have a dataset of 10 images, and the model's predictions for each image are as follows:

Image	True Label	Top 5 Predicted Labels
1	Cat	[Dog, Cat, Rabbit, Fish, Bird]
2	Dog	[Cat, Dog, Horse, Bird, Fish]
3	Rabbit	[Fish, Bird, Cat, Rabbit, Dog]
4	Bird	[Dog, Horse, Bird, Cat, Rabbit]
5	Fish	[Fish, Cat, Dog, Rabbit, Bird]
6	Cat	[Rabbit, Dog, Bird, Fish, Cat]
7	Dog	[Dog, Cat, Rabbit, Bird, Fish]
8	Bird	[Horse, Fish, Cat, Dog, Bird]
9	Rabbit	[Dog, Rabbit, Bird, Fish, Cat]
10	Fish	[Rabbit, Bird, Cat, Fish, Dog]

- To calculate Acc@5, you use the following formula:

$$\text{Acc@5} = \frac{\text{Number of correct predictions (true label in top 5)}}{\text{Total number of images}} \times 100$$

$$\text{Acc@5} = \frac{10}{10} \times 100 = 100\%$$

1. Adaptive Pooling

- Adaptive pooling allows the output size of the pooling layer to be specified, regardless of the input size. It dynamically adjusts the pooling operation to produce a fixed-size output from varying input sizes.

- For example, you can specify that the output should be 1×11 \times 11×1, 2×22 \times 22×2, etc.

- The goal of adaptive pooling is to allow flexibility in handling inputs of different sizes while ensuring that the output has a specified fixed size. This is particularly useful in tasks like object detection or segmentation, where input dimensions may vary.

2. Link: Dilation

3. GFLOPS stands for **Giga Floating Point Operations Per Second**. It is a measure of a computer's performance, particularly in tasks that involve floating-point calculations, such as scientific computations, graphics rendering, and deep learning tasks.

- **Giga**: Refers to a billion. In this context, 1 Giga equals 10910^9109 or 1,000,000,000.

- **Floating Point Operations**: These are calculations involving floating-point numbers, which are numbers that have decimal points and can represent a wide range of values. They are commonly used in complex mathematical calculations.

- **Operations Per Second**: This indicates how many of these floating-point operations can be performed in one second.